KB금융그룹 | 국민의 평생 금융파트너

K-디지털 트레이닝

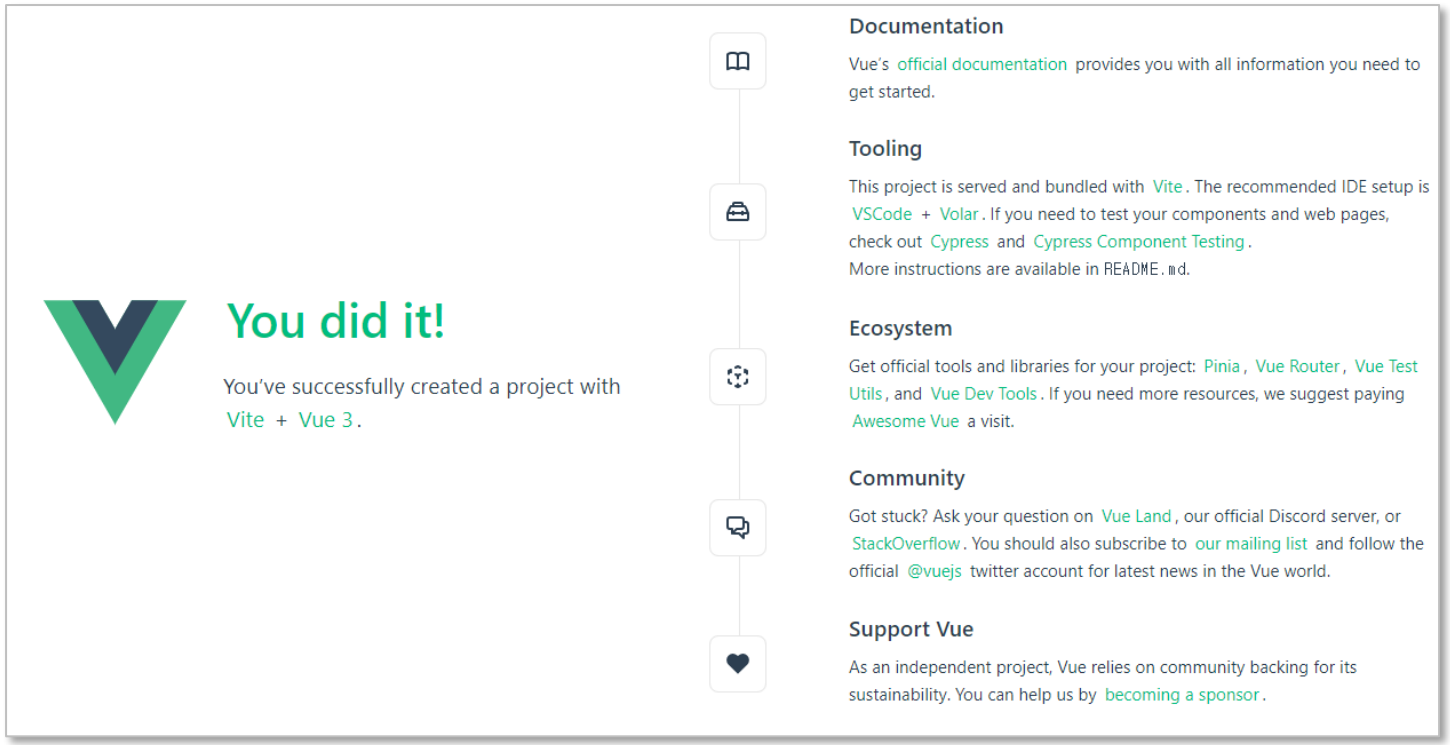
# 단일 파일 컴포넌트를 이용한 Vue 애플리케이션 개발

[KB] IT's Your Life

KB국민은행

# 단일 파일 컴포넌트

- 다음 조건을 만족하는 단일 파일 컴포넌트 프로젝트를 생성하고 실행하세요.
  - 프로젝트명: test-vue-app
  - 관련 패키지 인스톨
  - 개발 모드로 실행



# 단일 파일 컴포넌트

## 프로젝트 만들기, 실행

- npm init vue test-vue-app
- cd test-vue-app
- npm install
- npm run dev

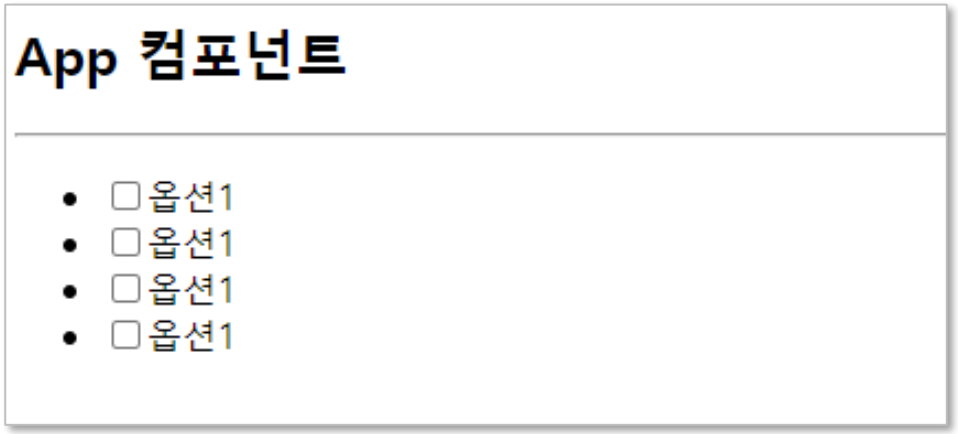
# 단일 파일 컴포넌트

- test-vue-app 프로젝트에 다음의 처리를 진행하세요.
  - src/main.js 수정
    - main.css 임포트 제거
  - src/components 폴더내 컴포넌트 모두 제거
  - src/App.vue 수정
    - 기존 임포트된 컴포넌트 모두 제거
    - 스타일 모두 제거

```
<template>
  <div>
    <h2>App 컴포넌트</h2>
  </div>
</template>

<script>
export default {
  name: 'App',
};
</script>
```

# 단일 파일 컴포넌트

- 다음 그림과 같이 출력되도록 다음 절차에 따라 코드를 완성하세요.
  - src/Components/CheckboxItem.vue 정의
  - CheckboxItem.vue를 **전역 컴포넌트로** 설정
  - App.vue에서 CheckboxItem 컴포넌트 4개 배치하여 사용



## src/components/CheckboxItem.vue

```
<template>
  <li><input type="checkbox" v-model="checked" />옵션1</li>
</template>

<script>
export default {
  name: 'CheckboxItem',
  data() {
    return {
      checked: false,
    };
  },
};
</script>
```

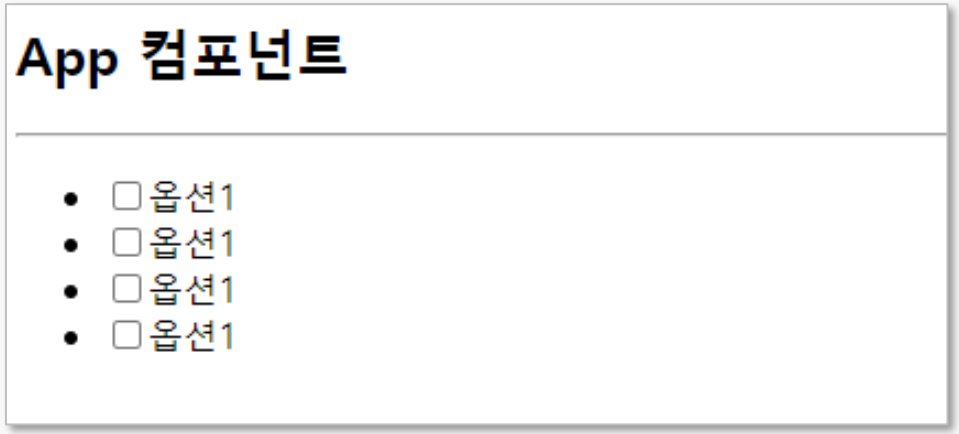
## src/main.js

```
import { createApp } from 'vue';
import App from './App.vue';
import CheckboxItem from './components/CheckboxItem.vue';

createApp(App)
    .component('CheckboxItem', CheckboxItem)
    .mount('#app');
```

# 단일 파일 컴포넌트

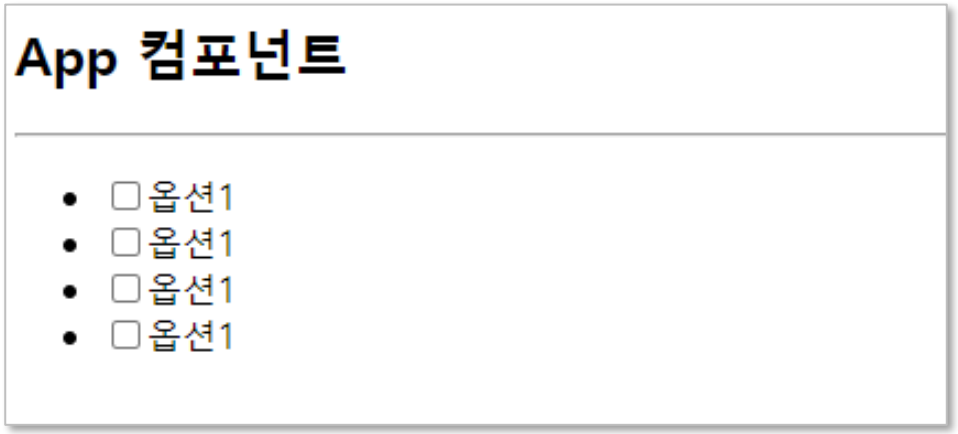
## src/App.vue

```
<template>
  <div>
    <h2>App 컴포넌트</h2>
    <hr />
    <ul>
      <CheckboxItem />
      <CheckboxItem />
      <CheckboxItem />
      <CheckboxItem />
    </ul>
  </div>
</template>

<script>

export default {
  name: 'App',
};
</script>
```



# 단일 파일 컴포넌트

- 앞 문제의 결과에서 다음 다음 절차에 따라 코드를 수정하세요.
  - 전역 CheckboxItem 컴포넌트 설정 취소
  - CheckboxItem.vue를 App.vue의 **지역 컴포넌트로** 설정
  - App.vue에서 CheckboxItem 컴포넌트 4개 배치하여 사용



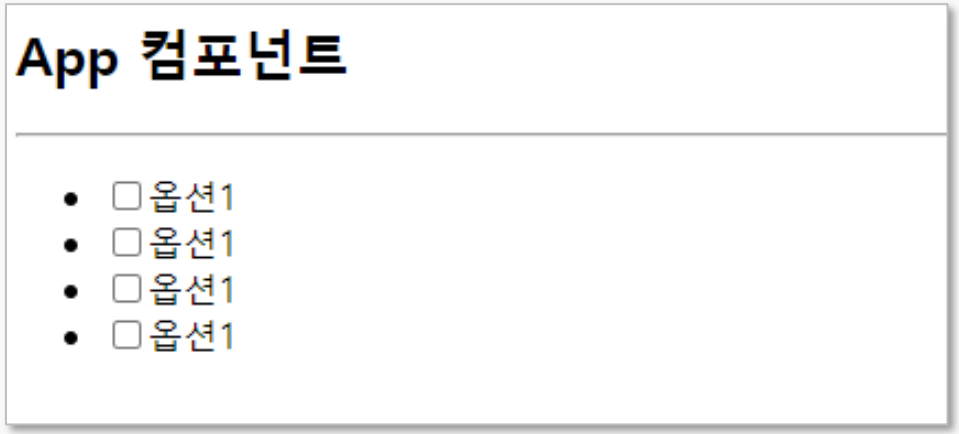
## src/main.js

```
import { createApp } from 'vue';
import App from './App.vue';

createApp(App)
  .mount('#app');
```

# 단일 파일 컴포넌트

## src/App.vue

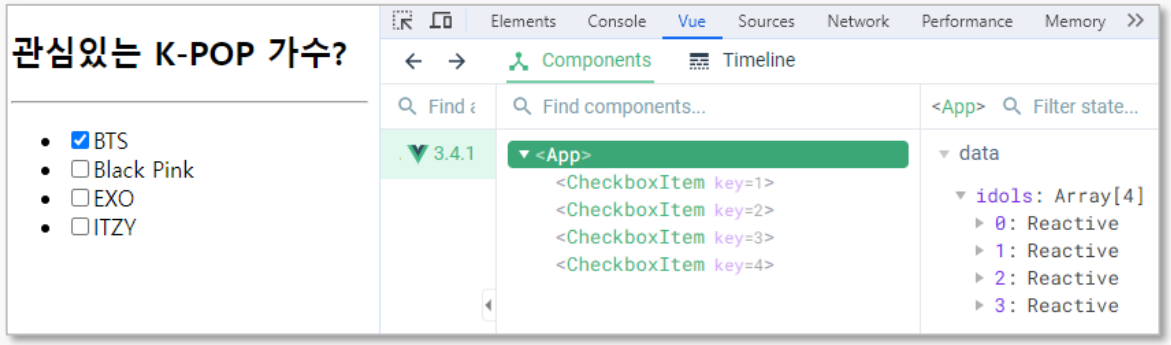
```
<template>
  <div>
    <h2>App 컴포넌트</h2>
    <hr />
    <ul>
      <CheckboxItem />
      <CheckboxItem />
      <CheckboxItem />
      <CheckboxItem />
    </ul>
  </div>
</template>

<script>
import CheckboxItem from './components/CheckboxItem.vue';

export default {
  name: 'App',
  components: { CheckboxItem },
};
</script>
```



# 단일 파일 컴포넌트

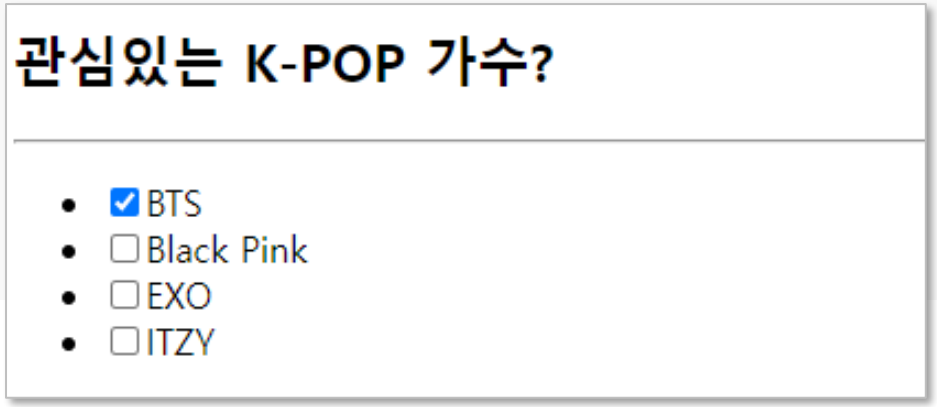
- App.vue에서 다음과 같은 데이터 옵션을 추가하고, 이 데이터 수 만큼 CheckboxItem을 생성한 후 데이터를 바인딩하세요.
  - CheckboxItem에 name, checked 속성을 추가하세요.
  - 크롬 Vue 개발 확장팩에서 데이터를 확인하세요

```
<script>
import CheckboxItem from './components/CheckboxItem.vue';
export default {
  name: 'App',
  components: { CheckboxItem },
  data() {
    return {
      idols: [
        { id: 1, name: 'BTS', checked: true },
        { id: 2, name: 'Black Pink', checked: false },
        { id: 3, name: 'EXO', checked: false },
        { id: 4, name: 'ITZY', checked: false },
      ],
    };
  },
};
</script>
```



## src/components/CheckboxItem.vue

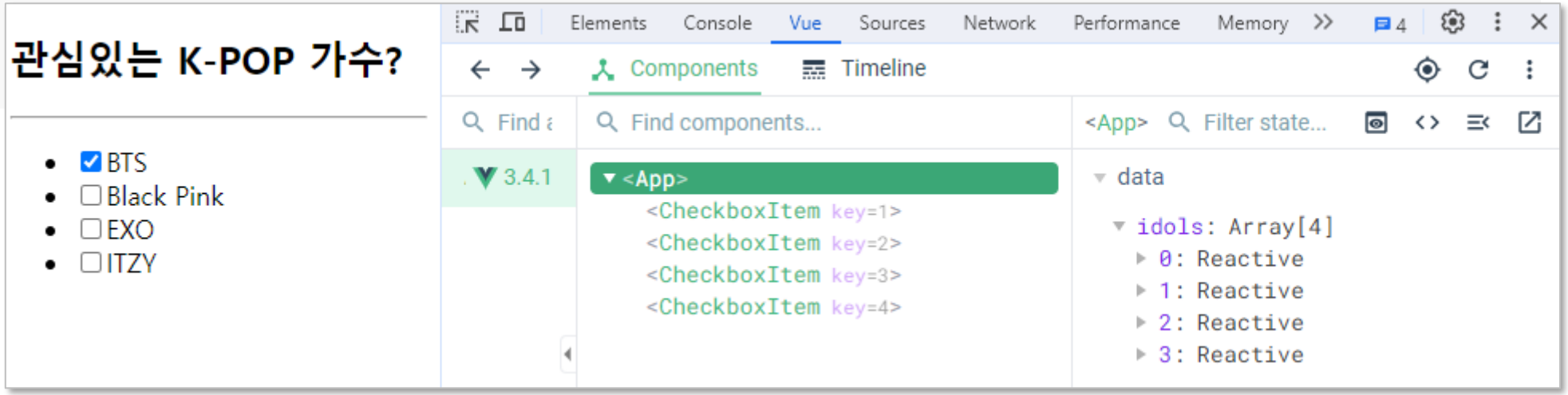
```
<template>
  <li><input type="checkbox" :checked="checked" />{{ name }}</li>
</template>

<script>
export default {
  name: 'CheckboxItem',
  props: ['name', 'checked'], // 속성명을 배열로 정의
};
</script>
```



# 단일 파일 컴포넌트

## src/App.vue

```
<template>
  <div>
    <h2>관심있는 K-POP 가수?</h2>
    <hr />
    <ul>
      <CheckboxItem
        v-for="idol in idols"
        :key="idol.id"
        :name="idol.name"
        :checked="idol.checked"
      />
    </ul>
  </div>
</template>
```



## src/App.vue

```
<script>
import CheckboxItem from './components/CheckboxItem.vue';
export default {
  name: 'App',
  components: { CheckboxItem },
  data() {
    return {
      idols: [
        { id: 1, name: 'BTS', checked: true },
        { id: 2, name: 'Black Pink', checked: false },
        { id: 3, name: 'EXO', checked: false },
        { id: 4, name: 'ITZY', checked: false },
      ],
    };
  },
};
</script>
```

- 배열 요소 하나를 idol이라는 속성으로 받아서 처리하도록 CheckboxItem2를 정의하고, App2.vue에서 운영해보세요.
  - App2.vue

```
<template>
  <div>
    <h2>관심있는 K-POP 가수?</h2>
    <hr />
    <ul>
      <CheckboxItem ____________________________________________/>
    </ul>
  </div>
</template>

<script>
import CheckboxItem from './components/CheckboxItem2.vue';
export default {
  name: 'App',
  …
};
</script>
```

## src/CheckboxItem2.vue

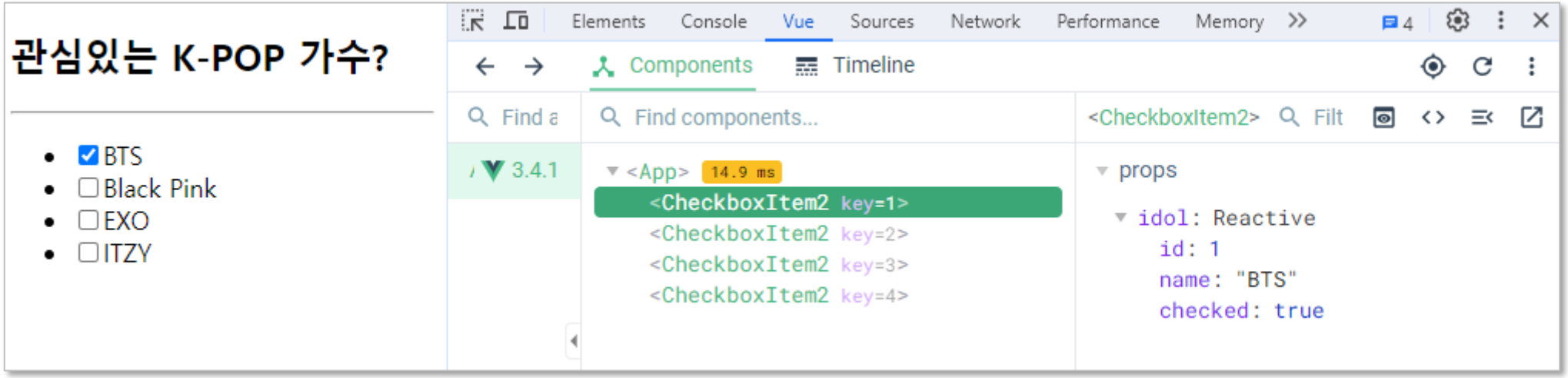
```
<template>
  <li><input type="checkbox" :checked="idol.checked" />{{ idol.name }}</li>
</template>

<script>
export default {
  name: 'CheckboxItem2',
  props: ['idol'],
};
</script>
```

## src/App2.vue

```
<template>
  <div>
    <h2>관심있는 K-POP 가수?</h2>
    <hr />
    <ul>
      <CheckboxItem v-for="idol in idols" :key="idol.id" :idol="idol" />
    </ul>
  </div>
</template>

<script>
import CheckboxItem from './components/CheckboxItem2.vue';
export default {
  name: 'App',
  …
};
</script>
```

## src/main.js

```
import { createApp } from 'vue';
// import App from './App.vue';
import App from './App2.vue';

createApp(App).mount('#app');
```



## 다음을 처리하도록 CheckboxItem을 수정하세요.

- 속성 목록
  - id는 Number 또는 String 타입만 가능
  - name은 String 타입
  - checked는 Boolean 타입이며, 기본값으로 false를 가지며, 옵션임(생략 가능)

## src/components/CheckboxItem.vue

```
<template>
  <li><input type="checkbox" :checked="checked" />{{ id }} - {{ name }}</li>
</template>

<script>
export default {
  name: 'CheckboxItem',
  props: {
    id: [Number, String], // Number 또는 String 타입
    name: String,         // String 타입
    checked: {
      type: Boolean,      // Boolean 타입
      required: false,    // 옵션
      default: false,     // 생략시, 기본값은 false
    },
  },
};
</script>
```

## 다음을 만족하는 InputName 컴포넌트를 정의하세요.

- 컴포넌트의 모양은 다음과 같이 구성한다.

이름: [ ] 이벤트 발신

- 이벤트 발신 버튼을 클릭하면 nameChangedHandler() 함수를 호출한다.
  - 이벤트 명은 'nameChanged'.
  - 이벤트 인자는 name
- App4.vue에서 이벤트로 전달된 데이터를 출력한다.

이름: 홍길동 이벤트 발신

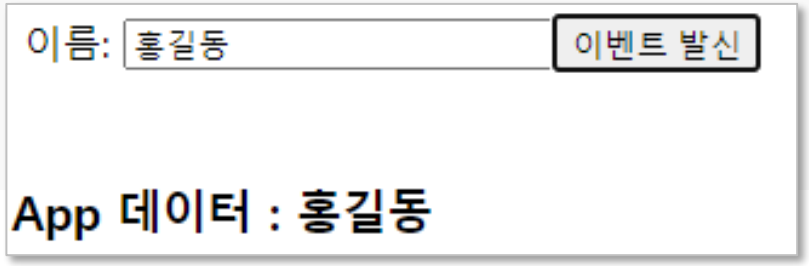
App 데이터 : 홍길동

## src/components/InputName.vue

```
<template>
  <div style="border: solid 1x gray; padding: 5px">
    이름: <input type="text" v-model="name" />
    <button @click="$emit('nameChanged', { name })">이벤트 발신</button>
  </div>
</template>

<script>
export default {
  name: 'InputName',
  data() {
    return { name: '' };
  },
};
</script>
```

## src/App4.vue

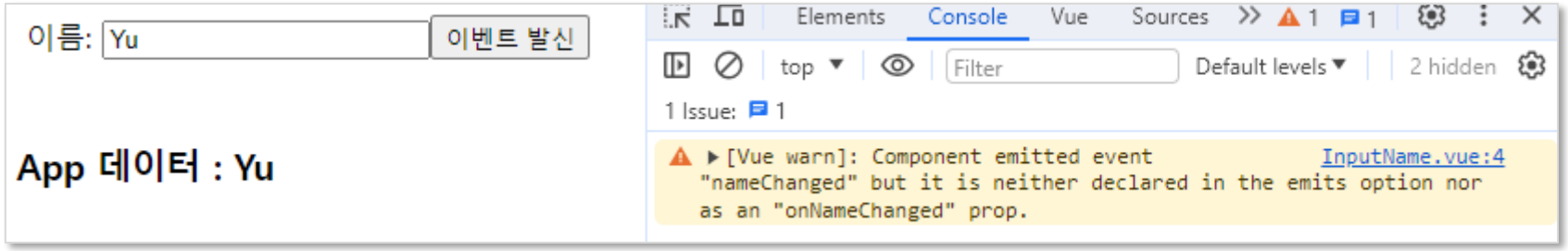
```vue
<template>
  <div>
    <InputName @nameChanged="nameChangedHandler" />
    <br />
    <h3>App 데이터 : {{ parentName }}</h3>
  </div>
</template>
```

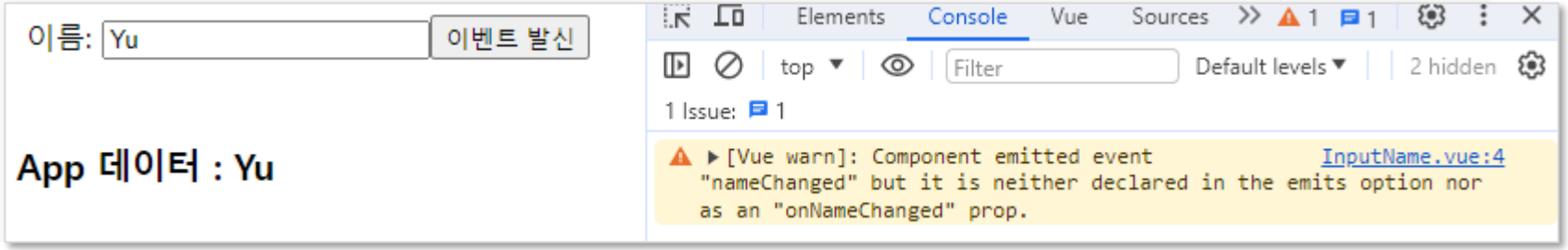



```vue
<script>
import InputName from './components/InputName.vue';

export default {
  name: 'App4',
  components: { InputName },
  data() {
    return {
      parentName: '',
    };
  },
  methods: {
    nameChangedHandler(e) {
      this.parentName = e.name;
    },
  },
};
</script>
```

## InputName 컴포넌트에 다음 조건을 추가하세요.

- namedChanged 이벤트에 유효성 검사를 추가함
  - name 값이 있어야 함
  - name 값의 타입은 문자열이어야 함
  - name 값의 길이가 3이상이여 함
- 유효성 검사에 실패한 경우 콘솔에 경고 메시지가 출력됨을 확인함



## src/InputName.vue

```
<template>…
</template>

<script>
export default {
  name: 'InputName',
  // emits: ['nameChanged1'], // nameChanged1 이외의 이벤트 발생시 경고창 출력
  emits: {
    namedChanged: (e) => {
      return e.name && typeof e.name === 'string' && e.name.trim().length >= 3
        ? true
        : false;
    },
  },
  data() {
    return { name: '' };
  },
};
</script>
```